



**UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE – CAMPUS NATAL
CURSO TECNOLOGIA DA INFORMAÇÃO**

RELATÓRIO TÉCNICO : Sistema de Gerenciamento de Biblioteca

Aluno: Nelson Ryan Silva de Vasconcelos

Matrícula:20250062976

Disciplina: Introdução a Técnicas de Programação (2025.2)

Projeto da Unidade 1: Sistema de Gerenciamento de Biblioteca Pt.2

Professor: Fernando Marques Figueira Filho

Data: 27/11/2025

Natal - RN

2025

1. Introdução e Contexto

O presente relatório detalha o processo de evolução do projeto da Unidade 1, expandindo o Sistema de Gerenciamento de Biblioteca para atender aos requisitos da Unidade 2. Enquanto a primeira versão focava na lógica básica de cadastro com limites fixos, o objetivo principal desta etapa foi implementar a **alocação dinâmica de memória**. O sistema foi reestruturado para permitir que o acervo cresça conforme a necessidade do usuário, sem desperdício inicial de recursos ou limites arbitrários. Além disso, a aplicação foi aprimorada com novas funcionalidades, como a busca específica por termos e a ordenação alfabética dos livros, aplicando conceitos mais avançados de manipulação de strings e modularização.

2. Análise Técnica

2.1. Metodologia e Ferramentas

Para a construção desta nova etapa, o ambiente de desenvolvimento permaneceu o mesmo (VS Code e GCC), mas a abordagem de codificação mudou significativamente com a inclusão de novos arquivos e bibliotecas:

- **Bibliotecas:** Inclusão da `<stdlib.h>` (para gestão de memória) e uso intensivo da `<string.h>` (para manipulação segura de textos).
- **Modularização:** O código, antes concentrado em um único arquivo, foi dividido em `main.c`, `biblioteca.c` e `biblioteca.h`. Essa separação é benéfica pois isola a lógica do sistema da interface com o usuário, facilitando a manutenção.

2.2. Aplicação dos Conceitos da Unidade 2

Os conceitos fundamentais da unidade foram aplicados da seguinte forma:

- **Ponteiros e Alocação Dinâmica:** A estrutura de vetores estáticos da Unidade 1 foi substituída por ponteiros. Utilizou-se a função `malloc` para criar o espaço inicial e `realloc` para expandir a capacidade do sistema automaticamente (dobrando o tamanho) sempre que o limite é atingido. Dessa forma, o programa tornou-se escalável, gerenciando a memória *Heap* do computador de forma eficiente.

- **Strings (Manipulação Avançada):** O tratamento de textos foi otimizado. Funções como `strlen` foram usadas para limpar o buffer de entrada (substituindo o laço manual usado anteriormente). Além disso, a função `strstr` permitiu implementar a busca de substrings (localizando partes de um título ou autor), e a função `strcmp` foi essencial para comparar a ordem alfabética dos registros.
- **Estruturas de Repetição Aninhadas:** Laços dentro de laços foram cruciais para a nova funcionalidade de ordenação. Implementou-se o algoritmo *Bubble Sort*, onde um laço externo percorre a lista e um laço interno compara elementos adjacentes, trocando-os de posição caso estejam fora de ordem alfabética.

3. Desafios, Soluções e Organização

3.1. Dificuldades Encontradas

Uma das principais dificuldades encontradas durante o desenvolvimento foi lidar com a sintaxe de **ponteiros para matrizes** e a precedência de operadores. Inicialmente, a tentativa de acessar os dados usando a sintaxe comum (`*titulo[i]`) acarretou em erros de segmentação, pois o compilador interpretava a ordem de acesso de forma incorreta. Outro desafio foi compreender a lógica de **ponteiros duplos** (**), necessária para que a função de cadastro conseguisse atualizar o endereço de memória da variável original na função `main` após uma realocação.

3.2. Soluções Implementadas

Para solucionar os desafios de sintaxe, o método adotado foi o uso explícito de parênteses, como em `(*titulo)[i]`, instruindo o programa a desreferenciar o ponteiro antes de acessar o índice do vetor. Quanto à alocação, implementou-se uma verificação de segurança: o `realloc` é feito primeiro em um ponteiro temporário. Somente se a operação for bem-sucedida (diferente de `NULL`), o ponteiro original é atualizado, evitando a perda de dados em caso de falha de memória.

3.3. Organização do Código

O projeto agora segue uma estrutura profissional sugerida na disciplina:

- **include/:** Contém o arquivo de cabeçalho (.h) com os protótipos, permitindo que os arquivos .c "conversem" entre si.
- **src/:** Contém a implementação das funções e o arquivo principal. Essa organização deixou a função main extremamente limpa, contendo apenas o fluxo de controle e a gestão de vida da memória.

4. Respostas às Perguntas Orientadoras

- **Quais conceitos da Unidade 2 foram aplicados e onde? A alocação dinâmica** foi aplicada na criação e expansão dos vetores de dados. As **strings** foram manipuladas nas funções de busca e ordenação. As **estruturas aninhadas** foram aplicadas exclusivamente no algoritmo de ordenação (*Bubble Sort*).
- **Como foram implementadas as estruturas de dados complexas (matrizes)?** Elas foram implementadas através de ponteiros para arrays (char (*p)[80]) alocados dinamicamente. Isso permite tratar um bloco de memória contínuo como se fosse uma matriz bidimensional, mantendo a facilidade de acesso via índices.
- **Qual a estratégia para gerenciamento de memória?** A estratégia adotada foi a de "crescimento dobrado". O sistema inicia com capacidade pequena e dobra de tamanho automaticamente quando necessário. Isso equilibra o desempenho (evitando chamar realloc a todo momento) com a economia de memória.
- **Como você garante que não há vazamentos de memória?** Para cada chamada de malloc realizada no início do programa, existe uma chamada correspondente de free no final da função main. Isso garante que, ao encerrar o programa, toda a memória alugada seja devolvida ao sistema operacional.
- **Quais vantagens a alocação dinâmica trouxe para seu projeto?** A principal vantagem foi remover a limitação da constante MAX_LIVROS da Unidade 1. O sistema agora é limitado apenas pela memória física do computador, tornando-o muito mais flexível para o usuário final.

5. Conclusão

O desenvolvimento desta segunda etapa permitiu atingir um nível de complexidade técnica superior. A transição para memória dinâmica não apenas resolveu as limitações do projeto anterior, mas também proporcionou um aprendizado profundo sobre como o computador gerencia seus recursos. O principal aprendizado obtido foi a importância de entender a aritmética de ponteiros e a organização modular, habilidades essenciais para desenvolver softwares robustos e escaláveis em C. O projeto agora possui uma base sólida para receber implementações de persistência de dados em arquivos na próxima unidade.